

Lab 7: Database Connectivity

ส่วนที่ 1: Software Design & Principles Explanation

สถาปัตยกรรมและ GRASP Patterns:

หน้าที่ของคลาส Entity คือ คลาสที่กำหนดว่าเกมจะเก็บข้อมูลอะไรบ้าง เช่น ชื่อเกม ราคา และประเภทส่วนลด ส่วนคลาส Repository คือคลาสที่ทำหน้าที่ติดต่อและเข้าถึงข้อมูลในฐานข้อมูล คลาส Service เป็นคลาสที่จัดการ Business Logic และการทำงานเกี่ยวกับ CRUD โดยเรียกใช้ Method จาก Repository รวมถึงเรียกใช้ Strategy เมื่อต้องมีการคำนวณส่วนลด และคลาส Controller คือคลาสที่รับ HTTP Request จาก Browser แล้วเรียกใช้ Service เพื่อประมวลผลและส่งข้อมูลกลับไปแสดงผล

การแบ่งหน้าที่แบบนี้สอดคล้องกับ GRASP Patterns โดย GameController ทำหน้าที่รับ Request จากภายนอกตาม Controller Pattern แต่ละคลาสถูกแยกให้มีหน้าที่เฉพาะของตัวเอง ทำให้เกิด High Cohesion และ Controller ไม่ติดต่อกับ Database โดยตรง แต่เรียกผ่าน Service และ Repository ทำให้ลดการพึ่งพากันระหว่างคลาสหรือเกิด Low Coupling นอกจากนี้ Service ยังเป็นตัวกลางระหว่าง Controller กับ Repository ซึ่งสอดคล้องกับหลัก Indirection

High-Level SOLID Principles:

การประยุกต์ใช้

SRP แยกแต่ละคลาสให้มีหน้าที่รับผิดชอบคนละส่วน

OCP ใช้ Strategy Pattern โดยออกแบบให้สามารถเพิ่ม Strategy ใหม่ได้ ถ้าในอนาคตมีส่วนลดประเภทอื่น โดยไม่ต้องแก้ไข Strategy เดิม

LSP DiscountContext สามารถรับ NoDiscountStrategy, StudentDiscountStrategy หรือ SeasonalSaleStrategy มาใช้แทนกันได้ เพราะทุกคลาส implement มาจาก DiscountStrategy และสามารถทำงานตามรูปแบบเดียวกันได้

ISP DiscountStrategy มีเฉพาะ Method ที่เกี่ยวข้องกับการคำนวณส่วนลด ไม่มี Method อื่นที่ไม่เกี่ยวข้อง ทำให้คลาสที่ implement ไม่ต้องมี Method ที่ตัวเองไม่จำเป็นต้องใช้

DIP DiscountContext พึ่งพา Interface DiscountStrategy แทนการผูกกับ StudentDiscountStrategy หรือ Strategy อื่นๆ ทำให้สามารถเปลี่ยน Strategy ที่นำมาใช้คำนวณส่วนลดได้ง่าย

Strategy Pattern:

เพราะมีส่วนลดอยู่หลายรูปแบบ ถ้านำการคำนวณส่วนลดทั้งหมดไปเขียนไว้ในคลาสเดียวกัน จะต้องใช้ if-else เพื่อเลือกส่วนลด และถ้ามีการเพิ่มส่วนลดใหม่ ก็จะต้องกลับมาแก้ไข

โค้ดเดิม จึงสร้าง Interface กลางคือ DiscountStrategy และให้ส่วนลดแต่ละแบบ implement DiscountStrategy เพื่อนำไปใช้ จากนั้น DiscountContext จะรับ Strategy ที่ต้องการมาใช้ในการคำนวณส่วนลด

ดังนั้น Strategy Pattern จึงช่วยแยก Algorithm การคำนวณส่วนลดแต่ละแบบออกจากกัน และสนับสนุนหลัก OCP เพราะสามารถเพิ่มรูปแบบส่วนลดใหม่ได้ โดยไม่ต้องกลับไปแก้ไข โค้ดของ Strategy เดิม

Layered Architecture:

เพราะถ้านำการทำงานทุกอย่างไปรวมไว้ใน Controller จะทำให้ Controller มีหน้าที่มากเกินไป และทำให้แก้ไข Code ได้ยาก จึงแบ่งการทำงานออกเป็น Layer ดังนี้

Presentation Layer เป็น Controller สนใจการรับ HTTP Request และส่ง Response

Business Layer เป็น Service สนใจการทำงานของ Business Logic

Data Access Layer เป็น Repository สนใจการเข้าถึงและจัดการข้อมูลใน Database

Database Layer เป็น PostgreSQL สำหรับจัดเก็บข้อมูล

การแบ่งแบบนี้ทำให้เกิด High Cohesion เพราะงานที่เกี่ยวข้องกันอยู่ในส่วนเดียวกัน และทำให้เกิด Low Coupling เพราะแต่ละ Layer ไม่จำเป็นต้องรู้รายละเอียดการทำงานของ Layer อื่น

Execution Flow:

เมื่อเปิด localhost Browser จะส่ง HTTP Request ไปที่ Controller จากนั้น Controller จะรับ Request และเรียกใช้ GameService โดย Service จะเรียก GameRepository เพื่อดึงข้อมูลจาก PostgreSQL

เมื่อ Database ส่งข้อมูลเกมกลับมา ข้อมูลจะถูกส่งผ่าน Repository กลับไปยัง Service และ Service จะนำข้อมูลไปประมวลผลตาม Business Logic หากมีการคำนวณส่วนลด จะเลือก DiscountStrategy ที่ต้องการ และ DiscountContext จะเรียกใช้ Strategy ที่เลือกเพื่อคำนวณราคาหลังส่วนลด

เมื่อประมวลผลเสร็จ ข้อมูลจะถูกส่งกลับไปยัง Controller จากนั้น Controller จะส่งข้อมูลไปที่ Thymeleaf เพื่อสร้าง HTML และส่งกลับไปยัง Browser เพื่อแสดงรายการเกมให้ผู้ใช้งานเห็น

ส่วนที่ 2: Code Implementation & Explanation

ระบบแบ่งโครงสร้าง Code ออกเป็นหลายส่วน ได้แก่ Entity, Repository, Strategy Package, Service และ Controller เพื่อให้แต่ละส่วนมีหน้าที่ชัดเจนและลดการพึ่งพากันโดยตรง

1. Entity

Game เป็น Entity ที่ใช้แทนข้อมูลเกมในระบบ เช่น ชื่อเกม ราคา ประเภทเกม หรือข้อมูลส่วนลด โดย Entity จะเชื่อมกับตารางใน PostgreSQL ผ่าน JPA

ตัวอย่างโครงสร้าง : Entity มีหน้าที่หลักในการกำหนดโครงสร้างของข้อมูลที่ระบบจะจัดเก็บใน Database

```
@Entity
@Table(name = "games")
public class Game {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    private String genre;
```

2. Repository

GameRepository ทำหน้าที่เป็น Data Access Layer สำหรับติดต่อกับฐานข้อมูล โดยใช้ Spring Data JPA

```
public interface GameRepository extends JpaRepository<Game, Long> {
}
```

การ extends JpaRepository ทำให้สามารถใช้ Method สำหรับ CRUD ได้ เช่น findAll(), findById(), save() และ deleteById() โดยไม่ต้องเขียน SQL เอง

3. Strategy Package

Strategy Package ใช้สำหรับแยก Algorithm การคำนวณส่วนลดออกจาก Business Logic หลักของระบบ มี Interface กลางคือ

```
public interface DiscountStrategy {
    double calculateDiscount(double price);
}
```

จากนั้นแต่ละรูปแบบส่วนลดจะ implement Interface นี้ เช่น

```
public class SeasonalSaleStrategy implements DiscountStrategy {

    @Override
    public double calculateDiscount(double price) {
        return price * 0.80;
    }
}
```

```
public class StudentDiscountStrategy implements DiscountStrategy {

    @Override
    public double calculateDiscount(double price) {
        return price * 0.90;
    }
}
```

และ DiscountContext จะรับ DiscountStrategy มาใช้ในการคำนวณ

```
public class DiscountContext {

    private DiscountStrategy strategy;

    public void setStrategy(DiscountStrategy strategy) {
        this.strategy = strategy;
    }

    public double executeStrategy(double price) {
        return strategy.calculateDiscount(price);
    }
}
```

ในส่วนนี้มีการใช้ Constructor Injection โดย DiscountStrategy ถูกส่งเข้ามาผ่าน Constructor ของ DiscountContext ทำให้ DiscountContext ไม่ผูกกับ Strategy ตัวใดตัวหนึ่งโดยตรง

4. Service

GameService ทำหน้าที่จัดการ Business Logic ของระบบ เช่น การดึงข้อมูล เพิ่ม แก้ไข หรือลบเกม รวมถึงการเรียกใช้ Strategy สำหรับคำนวณส่วนลด

เช่น : ในส่วนนี้ใช้ Constructor Injection โดย Spring จะส่ง Object ของ GameRepository เข้ามาให้ GameService ผ่าน Constructor ทำให้ GameService ไม่ต้องสร้าง Repository เองด้วย new GameRepository() และช่วยลดการผูกติดกันของคลาส

```
@Service
public class GameService {

    private final GameRepository gameRepository;

    public GameService(GameRepository gameRepository) {
        this.gameRepository = gameRepository;
    }

    public List<Game> getAllGames() {

        List<Game> games = gameRepository.findAll();

        // คำนวณราคาสุทธิของแต่ละเกม
        for (Game game : games) {
            game.setFinalPrice(calculateFinalPrice(game));
        }

        return games;
    }
}
```

5. Controller

GameController ทำหน้าที่รับ HTTP Request จาก Browser และเรียกใช้ GameService

```

@Controller
@RequestMapping("/games")
public class GameController {

    private final GameService gameService;

    public GameController(GameService gameService) {
        this.gameService = gameService;
    }

    @GetMapping
    public String listGames(Model model) {
        model.addAttribute("games", gameService.getAllGames());
        return "games/list";
    }
}

```

ในส่วนนี้ใช้ Constructor Injection โดย Spring จะส่ง GameService เข้ามาผ่าน Constructor ของ GameController, Controller จึงไม่จำเป็นต้องสร้าง Service เอง และสามารถเรียกใช้ Method ของ Service เพื่อประมวลผลข้อมูลได้

Dependency Injection และ Constructor Injection

Dependency Injection คือการที่ Object ไม่สร้าง Dependency ที่ตัวเองต้องใช้ขึ้นมาเอง แต่ให้ Spring เป็นผู้สร้างและส่ง Object ที่ต้องการเข้ามาให้

ตัวอย่าง Flow ของ Dependency : GameController → GameService → GameRepository โดย GameController ต้องใช้ GameService, GameService ต้องใช้ GameRepository, Spring จะสร้าง Object เหล่านี้และ Inject ผ่าน Constructor

```

public GameController(GameService gameService) {
    this.gameService = gameService;
}

```

```

public GameService(GameRepository gameRepository) {
    this.gameRepository = gameRepository;
}

```

ข้อดีของ Constructor Injection คือ Dependency ของแต่ละคลาสมีความชัดเจน ลดการผูกติดกันของคลาส สามารถเปลี่ยน Implementation ได้ง่าย และช่วยให้เขียน Unit Test ได้สะดวกขึ้น

ส่วนที่ 3: Web Application & Database Screenshots

หน้าจอเพิ่มเกมใหม่ (Create) ที่กำลังกรอกข้อมูลที่มีรหัสนักศึกษา + Section:

Game Catalog CP353002 — Lab 7

← กลับไปรายการเกม

+ เพิ่มเกมใหม่

กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม

Elden Ring (673380592-3 SEC 3)

แนวเกม (Genre) แพลตฟอร์ม (Platform)

Action RPG PC / PS5

คะแนน (Rating) ราคาปกติ (บาท) % ส่วนลด (Strategy)

9.8 1790 ส่วนลดนักศึกษา (10%)

วันวางจำหน่าย (Release Date)

02/25/2022

บันทึก ยกเลิก

หน้าจอแสดงรายการเกมทั้งหมด (Read) ที่เห็นแถบแจ้งเตือนสีเขียว สำเร็จ และข้อมูลเกมที่มีรหัสนักศึกษาในตาราง:

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

เพิ่มเกมใหม่

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชัน (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Overcooked! 2	Simulation	PC	★ 9.5	07/08/2018	ส่วนลดเทศกาล (20%)	423.20 529.00	
2	Elden Ring (673380592-3 SEC 3)	Action RPG	PC / PS5	★ 9.8	25/02/2022	ส่วนลดนักศึกษา (10%)	1611.00 1790.00	

ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

หน้าจอแก้ไขเกม (Update) แสดงฟอร์มแก้ไขข้อมูลเกม:

Game Catalog

CP353002 — Lab 7

← กลับไปรายการเกม

แก้ไขข้อมูลเกม

แก้ไขข้อมูลเกม: Elden Ring (673380592-3 SEC 3)

ชื่อเกม

Elden Ring (673380592-3 SEC 3)

แนวเกม (Genre)

Action RPG

แพลตฟอร์ม (Platform)

PC / PS5

คะแนน (Rating)

9.8

ราคาปกติ (บาท)

1790.0

% ส่วนลด (Strategy)

ส่วนลดเทศกาล (20%)

วันวางจำหน่าย (Release Date)

mm/dd/yyyy

อัปเดต

ยกเลิก

← → ↺

localhost:8080/games

🔍 ☆ 📱 School

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

📁 รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

➕ เพิ่มเกมใหม่

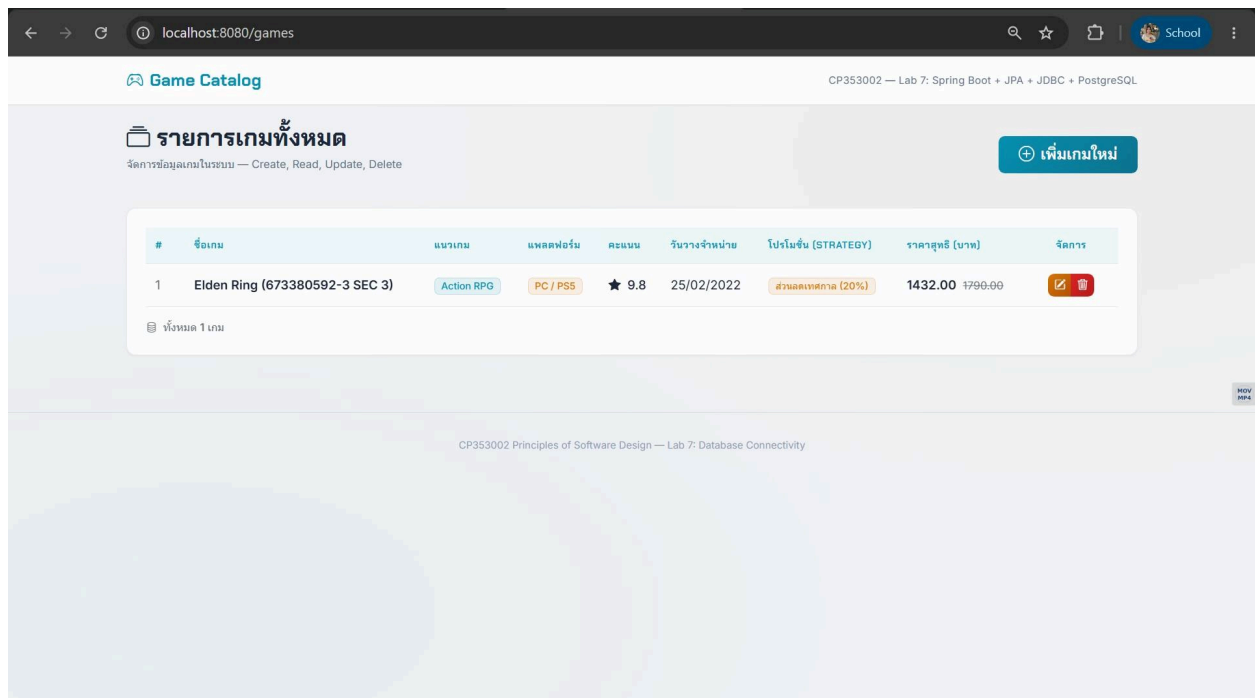
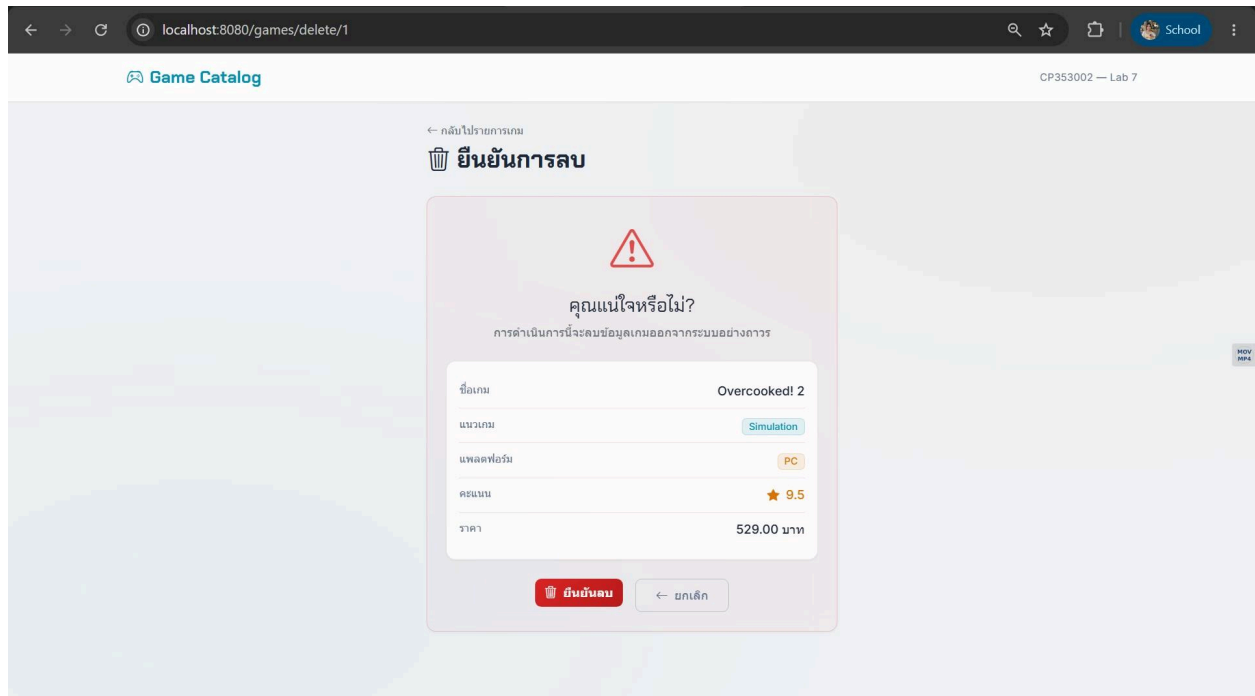
#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Overcooked! 2	Simulation	PC	★ 9.5	07/08/2018	ส่วนลดเทศกาล (20%)	423.20 529.00	✎ 🗑
2	Elden Ring (673380592-3 SEC 3)	Action RPG	PC / PS5	★ 9.8	25/02/2022	ส่วนลดเทศกาล (20%)	1432.00 1790.00	✎ 🗑

📄 ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

NOV 04

หน้าจอยืนยันลบ + ผลลัพธ์หลังลบ (Delete):



หน้าจอ Database (pgAdmin หรือ terminal psql) แสดงข้อมูลจริงในตาราง games ที่มีรหัสนักศึกษานั่นที่กอยู่:

ก่อนลบ

```
PS C:\Users\phava> & "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -d lab7demo -p 5433
Password for user postgres:

psql (17.10)
Type "help" for help.

lab7demo=# SELECT * FROM games;
 id | discount_type | genre      | platform | price | rating | release_date | title
-----+-----+-----+-----+-----+-----+-----+-----
  1 | SEASONAL      | Simulation | PC       |  529 |   9.5 | 2018-08-07   | Overcooked! 2
  3 | SEASONAL      | Action RPG | PC / PS5 | 1790 |   9.8 | 2022-02-25   | Elden Ring (673380592-3 SEC 3)
(2 rows)
```

หลังลบ

```
PS C:\Users\phava> & "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -d lab7demo -p 5433
Password for user postgres:

psql (17.10)
Type "help" for help.

lab7demo=# SELECT * FROM games;
 id | discount_type | genre      | platform | price | rating | release_date | title
-----+-----+-----+-----+-----+-----+-----+-----
  1 | SEASONAL      | Simulation | PC       |  529 |   9.5 | 2018-08-07   | Overcooked! 2
  3 | SEASONAL      | Action RPG | PC / PS5 | 1790 |   9.8 | 2022-02-25   | Elden Ring (673380592-3 SEC 3)
(2 rows)

lab7demo=# SELECT * FROM games;
 id | discount_type | genre      | platform | price | rating | release_date | title
-----+-----+-----+-----+-----+-----+-----+-----
  3 | SEASONAL      | Action RPG | PC / PS5 | 1790 |   9.8 | 2022-02-25   | Elden Ring (673380592-3 SEC 3)
(1 row)
```